Anisotropic Growth Vertex Model Feature Documentation

Avik Mondal

May 19, 2022

Contents

1 Preface

Most of what goes into the vertex model requires some kind of theoretical analysis before we decide on what gets coded. There's a few documents like this already: the document describing how anisotropic division are implemented and the document that includes all the work to figure out how to make the box grow anisotropically. I will include in this section a brief description of the two vertex models that I have made as of May 2022.

2 AGVertex Model

This is the "Anisotropic Growth Vertex Model." This was so labelled because it inherits all the features of Meryl's original Vertex Model (Vertex Model Base) minus the box update

equations. Instead, it features the updated NPT ensemble inspired algorithm described in "adjusting box size notes" (originally adapted from Carolina Baruffi's paper, *Application of an Overdamped Langevin Dynamics to the study of microstructural evolutions in crystalline materials.* Engineering Sciences[physics]. SORBONNE UNIVERSITY) as well as a few different methods of mitosis with divisions in preferred directions. As of May 2022, no research has really been conducted with this model.

3 AGPerimVertex Model

This is the vertex model that mimics the traditional vertex model which has a quadratic potential for the perimeter. It is called "AGPerimVertex" because

4 Backtracker in the AGPerimVertex Class

By the time anyone else has to read this, hopefully there is some other documentation or even a paper on the vertex model. To be brief, we hope to write a paper about how the use of A_0 s a parameter in the vertex model is spurious because it determines nothing, including the location of the phase transition. The suggested alternative to the typical shape index parameter is $\frac{p_0}{\langle A \rangle}$. In any case, this project required equilibrating the vertex model to very close to a "true" equilibrium, where the change in vertex model energies between timesteps are on the order of 10^{-15} or less. I found in developing this code that the default integrator would not dependably converge to this "true" equilibrium without help because the timesteps, set either as a constant in the input file or by the adaptive timestepper, can sometimes be too big. This requires a subsection to explain carefully.

4.1 Overshooting the Equilibrium

Citation: Strogatz, Nonlinear Dynamics and Chaos

It helps to imagine an overdamped harmonic oscillator as an analogy for the time integration of the vertex model. This can also help us to illustrate the problem that necessitates a "backtracker." The energy can be written down as $U = \frac{1}{2}K(x - x^{(0)})^2$, which then results in force of $F = -K(x - x^{(0)})^2$. The vertex model is overdamped, so we choose overdamped dynamics and write down an update equation of :

$$\dot{x} = -\frac{K}{\gamma}(x - x^{(0)})$$

If we were simulating this, we would need a difference equation as opposed to a differential equation so let's instead write:

$$x_{n+1} = -\frac{K}{\gamma}(x_n - x^{(0)})$$

I wrote out the preferred position with a superscript so that the subscripts could be used to label the index of the positions in the integration ¡– This is a terrible explanation, if you have time, think about how to write this better. Also, this equation needs a δt; here, I have set $\delta t = 0$.

This equation illustrates the need for a backtracking equation. If I were simulating this harmonic oscillator, I assert that close to equilibrium, the above update equation can result in a timestep that *increases* the energy instead of decreasing it, as gradient descent

should. Chapter 10.1 in Strogatz shows us how to prove this. For discrete dynamic systems where we define:

$$x_{n+1} = f(x_n),$$

one can determine whether the system will converge or diverge by taking the derivative of f with respect to x_n and evaluating it at a specific location. If that gives you a number whose magnitude is greater than one, the function will diverge. If it is less than on, the function will converge. For us, we can see that the derivative of f is $-\frac{K}{\gamma}$, which is independent of position and leads to a divergent dynamical system when $K > \gamma$.

So what does this have to do with the vertex model? It simply shows that it is quite possible that with too large a timestep, you can get cases where x_{n+1} diverges for a harmonic oscillator. While the vertex model is much more complicated, this simpler toy system could illustrate why I have observed an equilibrium overshooting.

4.2 The Backtracker

In pseudocode:

1. Check if energy increased. If yes, proceed with the following list. If no, skip the following.
2. If timestep is already smaller than the mininum timestep (this threshhold can be defined within the model), than call the tissue equilibrated.
3. Set positions back to those of previous timestep (Meryl already wrote code for this for the adaptive timestepper)
4. Set flags back to those of previous timestep
5. Set current time to previous time
6. Set current free energy value to past free energy
7. Reduce timestep by factor of choice.

5 Equilibrium in the Vertex Model

This section will cover how equilibrium is determined and set in the AGPerimVertex model. Many of "knobs" that exist in the AGPerimVertex were not needed and thus not implemented into the AGVertex model, though they probably could be moved there with little to no stress (but an hour or so of some coding and rearranging).

Equilibrium in the vertex model is defined by looking at the slope of the vertex model's nondimensionalized energy as a function of time. This energy ϵ can be defined as:

$$\epsilon = \frac{\sum_i E_i}{NKA_0^2}$$

where K has units of energy over area squared. To calculate the slope, I fit a linear regression curve to the last N timesteps, where N is a parameter in the AGPerimVertex model called "energyListSize." I use the slope of the regression curve to define equilibrium by comparing it to a variable called "equilibriumtolerance." This is set in the input file.

6 Changing the Vertex Model Energy

Outside of the general computer science of properly implementing (or not implementing) inheritance, the biggest thing to remember about changing the vertex model is to check the functions that implement T1's, make new edges, make new cells, and check the stability of functions in the vertex model. A mention of this should be made in the vertex model tutorial section.

7 Lloyd Iteration

This is a feature Jacob included in the MATLAB code for his work on hyperuniformity. The Lloyd iteration iteratively takes a packing and makes it more hyperuniform. During every iteration, the algorithm takes the Voronoi packing and moves the seed points to the center of mass (or centroid) of their corresponding cells. The following paper is the one Jacob listed as a good source for more detail:

Klatt MA, Lovrić J, Chen D, et al. Universal hidden order in amorphous cellular geometries. Nat Commun. 2019;10(1):811. Published 2019 Feb 18. doi:10.1038/s41467-019-08360-5

It's is worth noting that the Lloyd seems to also have a paper from 1982 so for details on the specific algorithm, that might be worth checking.

We employ the Lloyd iteration with almost every vertex model run because it gives us more regular packings. We usually restrict the number of iterations to 10 or less. However, less than 4 or 5 iterations has resulted in oscillating T1's and strange packings that retain triangular or even quadrilateral cells.

8 Oscillating T1 calculation

Oscillating T1's are the only way that one can get oscillations in the vertex model, given the dissipative dynamics. They are a problem when they result in an infinite loop: a) Tissue shrinks an edge to zero length - if the vertex model sees an edge smaller than a set min length, it does this. b) the model resolves a new edge back to the way the old edge was configured to a set new edge length c) Repeat a) and b) indefinitely. There are a few things necessary to avoid this problem. First, you need to check if fourfold vertices are stable - I talk about this in the next section. Second, there is a question of how you set the length of new edge in order to avoid T1 oscillation. Depending on the dynamics of the edge you are dealing with, you can imagine that there is an (stable) equilibrium edge length (if you lengthen the edge it shrinks; if you shrink it, it lengthens). This would be set by the specific configuration of your vertex model at a given timestep. If that equilibrium length is smaller than your new edge length and minimum length, you can expect to have oscillating T1's. This is because your new edge length and your minimum length would be located in the region where the dynamics push the edge to shrink toward the equilibrium, to something less than the minimum edge length. If your new edge length is smaller than your minimum edge length, regardless of some kind of equilibrium edge length, you'll have oscillating T1's. All of this to say, there are a few important things to mention:

1. Make your new edge length greater than your minimum edge length

2. Make both your new edge length and minimum edge length small (where small is relative to the length scale of cells in your model, such as $\sqrt{A_0}$ or $\sqrt{\langle A \rangle}$.

9 Updating Stability in Vertex Model

One of the problems that I ran across in creating the AGPerimVertex model was dealing with T1's with the new tension potential (quadratic in perimeter as opposed to linear). There's not much that's necessary to say here, especially since Meryl wrote an entire paper about this ("Vertex stability and topological transitions in vertex models of foams and epithelia" in Eur. Phys. J. E (2017)). Equation 22 in that paper gives a general stability condition (the fourfold vertex will be stable if ...) of the form:

$$\Gamma_\delta > \frac{\mathcal{F}}{2}$$

In that paper, Γ_δ refers to the constant tension on the edge that was shrunk down to a fourfold vertex. For the model that instead has $(P - P_0)^2$, we can recognize that Γ_δ can be replaced with $\Gamma(\Delta P)$. Γ is the perimeter modulus of the vertex model, assuming that the AGPerimVertex model's free energy is written as:

$$E = \sum_{\text{cells } i} K(A_i - A_0)^2 + \Gamma(P_i - P_0)^2.$$

ΔP_δ is the difference in perimeter between the two cells adjacent to the shrunken edge δ. $\mathcal{F}$ is the total forces on the newly created fourfold vertex ignoring forces that are defined parallel or perpendicular to the shrunken edge (Meryl's paper explains this in detail). So the change we have to make to the stability is:

$$\Delta P_\delta > \frac{\mathcal{F}}{2\Gamma}$$

Which indicates that there is a bound on the perimeter difference of the two adjacent cells that determines the stability of four fold vertices.

Meryl's paper ultimately showed that in the constant tension vertex model (the base model and AGVertex) cannot have stable fourfold vertices. It appears that the perimeter squared model is capable of having fourfold vertices - I have observed them and I checked that the stability condition is met and that the T1 is not responsible for increasing the overall energy of the packing.

10 Common Problems

- Updating Hamiltonian and forgetting to adjust T1's
- Forces growing too fast, particularly at the beginning – this one is not one that I have studied in depth. I can say that we had to scale the damping coefficient in the growing box model so that the tissue didn't break
- Precision Issues: Make sure that you write output files with large enough precision. When you need to equilibrate tissues very precisely, the precision error makes a huge difference

- Oscillating T1's - this problem is typically solved by evaluating the stability
- The code is not reading in your new variables. Double check that you've compiled your code correctly if you are working in Great Lakes, or that you've compiled the write start up project in C++ if you're using Visual Studios.